Run SELECT * FROM users;

Query | Query History

```
1  select * from users;
```

Data Output | Messages | Notifications

	id [PK] integer	email character varying	passwordHash character varying
1	1	auth.demo@example.com	$2b$12$TKgtL7e8E5rqrE5znRH28OieQQCMwvN0cXl5DSyIzzxi26LeAIoky
2	2	auth.postman.demo@example.com	$2b$12$Bl7nKzJl0RWKjlv/XMzvHulkoWCxi5hpUZE6jF0hapIflW47gHOci
3	3	auth.postman.evidence@example.com	$2b$12$R8gzx8ISG5nzNAJvibY78.mUWqjxZXOdgeIXzU.nGUWiUI0fIXAXi
4	4	auth.postman.evidence.run2@example.com	$2b$12$gf2EwWFQBkAuO1MZdbXmY.kO8.iELNKA9BpvrGJ3viMBWFk8cd…
5	5	auth.postman.final@example.com	$2b$12$.AxdPPRJuOGwPqrilKDhnOpNHAVKQZuNUpIkeTdxX4/YZscwKiM…
6	6	auth.postman.demo2@example.com	$2b$12$7e1xMLxAYyCLa/Y5iAs42uUECnsX8nSaJl2VPFCSz1suwTVH2.6…

Register new account

POST {{baseUrl}} /auth/register — Send

Docs | Params | Authorization | Headers 10 | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

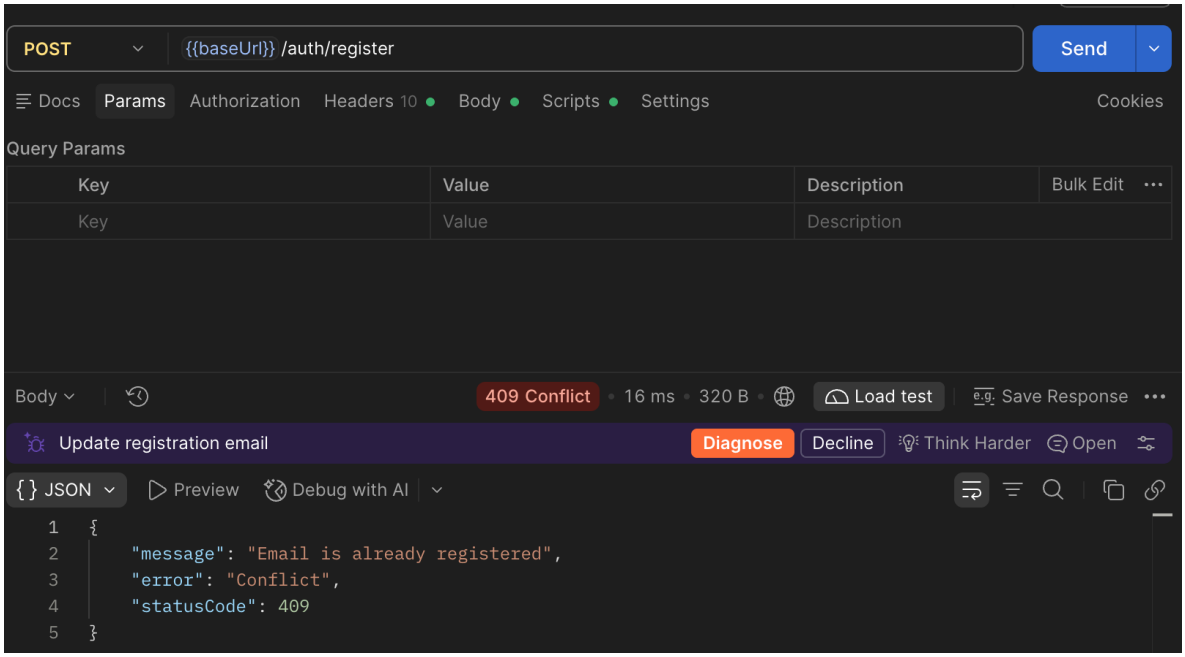
```
{
  "email": "auth.postman.demo2@example.com",
  "password": "{{password}}"
}
```

Body | 201 Created | 358 ms | 291 B | Load test | Save Response

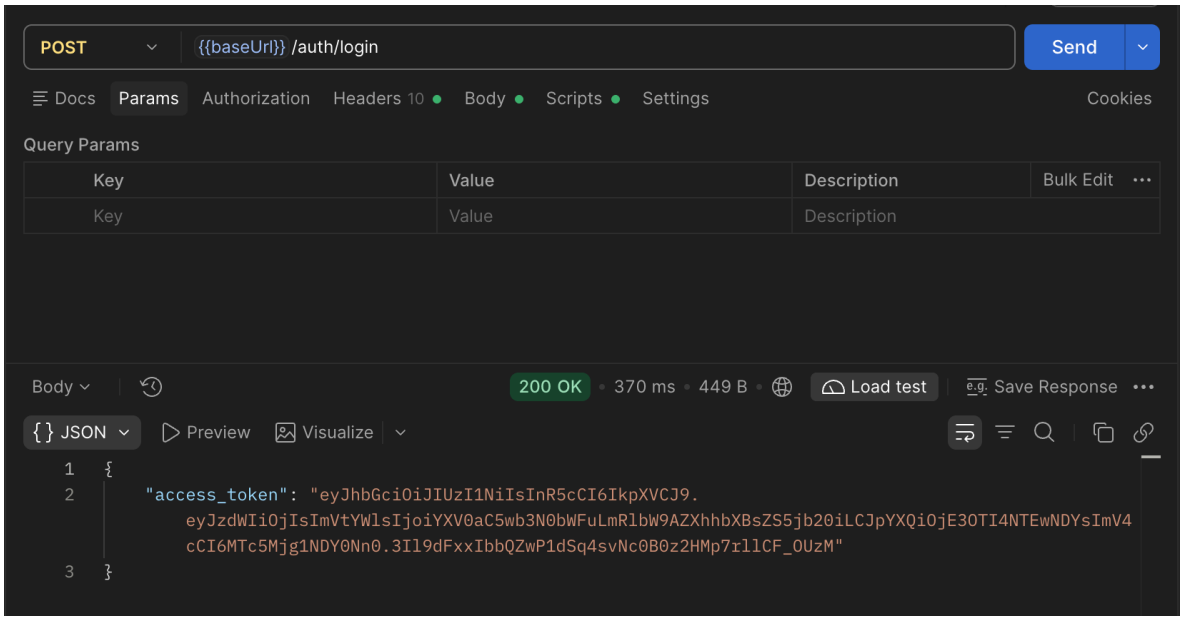
{ } JSON | Preview | Visualize

```
{
    "id": 6,
    "email": "auth.postman.demo2@example.com"
}
```

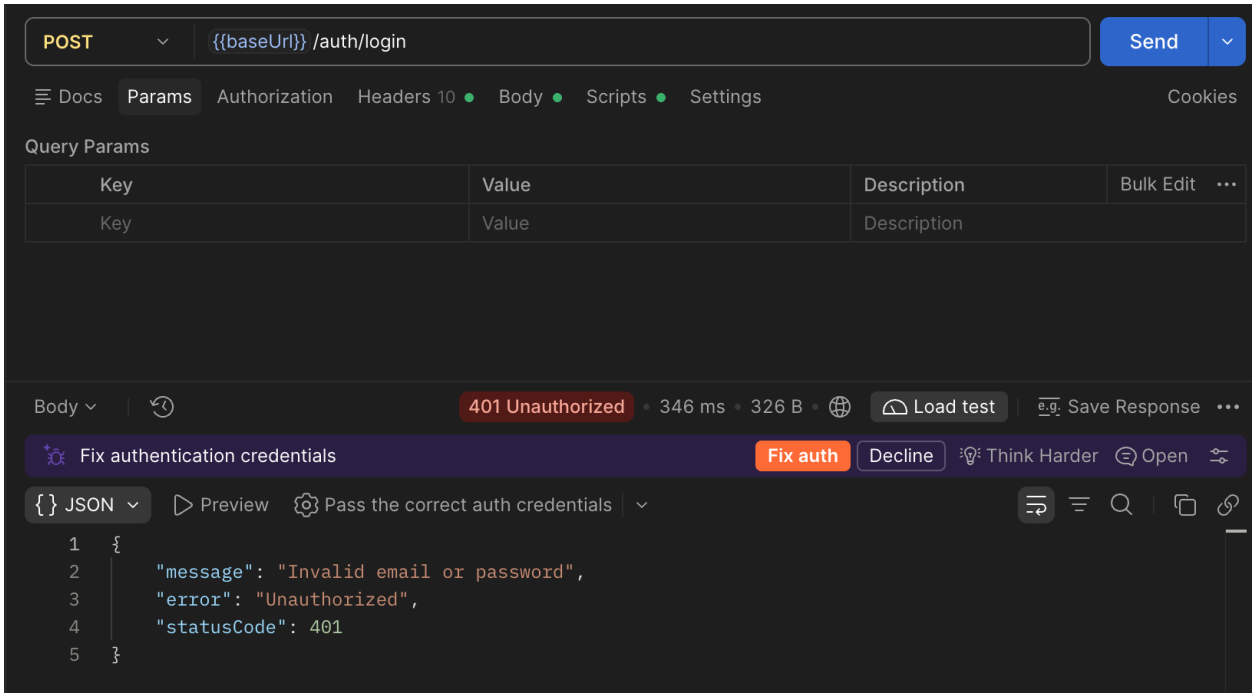
Register duplicate email



Login with correct password



Login with wrong password



main.ts

```ts
import { Logger, LogLevel, ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import type { NextFunction, Request, Response } from 'express';
import { AppModule, ObserveInstrument } from './app.module.js';

async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    instrument: ObserveInstrument,
    logger: getLogLevels(),
  });
  const requestLogger = new Logger('HTTP');
  app.useGlobalPipes(new ValidationPipe());
  app.use((request: Request, response: Response, next: NextFunction) => {
    const startedAt = Date.now();
    response.on('finish', () => {
      requestLogger.log(
        `${request.method} ${request.path} ${response.statusCode} ${Date.now() - startedAt}ms`,
      );
    });
    next();
  });
  await app.listen(process.env.PORT ?? 3000);
}

function getLogLevels(): LogLevel[] {
  const levels: LogLevel[] = ['error', 'warn', 'log'];
  if (process.env.NODE_ENV !== 'production') {
    levels.push('debug');
  }
  return levels;
}

await bootstrap();

```